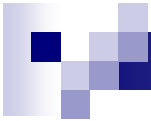


Группы процессов и коммуникаторы

- **Группа** – упорядоченный набор процессов.
- Каждый процесс в группе имеет свой ранг. Ранги в группе начинаются всегда с 0.
- **Коммуникатор** – механизм создания самостоятельной коммуникационной среды
- Коммуникатор включает *группу* и *контекст*
- **Контекст** – свойство коммуникатора, которое позволяет разделить коммуникационное пространство. Сообщение, отправленное в рамках одного контекста, не может быть получено в рамках другого



Создание группы процессов

- Получить группу заданного коммуникатора можно используя следующую функцию:

```
int MPI_Comm_group ( MPI_comm comm, MPI_Group *group )
```

- Создание новой группы **newgroup** из существующей группы **oldgroup**, которая будет включать в себя **n** процессов, ранги которых перечисляются в массиве **ranks**

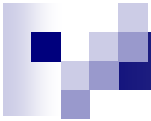
```
int MPI_Group_incl( MPI_Group oldgroup,  
                    int n,  
                    int *ranks,  
                    MPI_Group *newgroup );
```



Создание группы процессов

- Создание новой группы **newgroup** из группы **oldgroup**, которая будет включать в себя **n** процессов, ранги которых не совпадают с рангами, перечисленными в массиве **ranks**:

```
int MPI_Group_excl( MPI_Group oldgroup,  
                    int n,  
                    int *ranks,  
                    MPI_Group *newgroup );
```



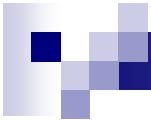
Создание группы процессов

- Над группами процессов можно выполнять те же операции, что и над множествами
 - создание новой группы **newgroup** как объединения групп **group1** и **group2**

```
int MPI_Group_union( MPI_Group group1,  
                    MPI_Group group2,  
                    MPI_Group *newgroup );
```

- создание новой группы **newgroup** как пересечения групп **group1** и **group2**:

```
int MPI_Group_intersection ( MPI_Group group1,  
                             MPI_Group group2,  
                             MPI_Group *newgroup );
```



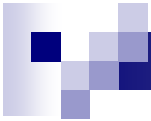
Создание группы процессов

- Над группами процессов можно выполнять те же операции, что и над множествами
 - создание новой группы **newgroup** как разности групп **group1** и **group2**:

```
int MPI_Group_difference ( MPI_Group group1,  
                           MPI_Group group2,  
                           MPI_Group *newgroup );
```

- Вычисление рангов процессов в группе **group2** на основании их рангов в группе **group1**

```
int MPI_Group_translate_ranks( MPI_Group group1,  
                               int n,  
                               const int ranks1[],  
                               MPI_Group group2,  
                               int ranks2[] );
```



Создание группы процессов

- Получение информации о группе процессов

- количество процессов в группе

```
int MPI_Group_size ( MPI_Group group, int *size );
```

- ранг текущего процесса в группе

```
int MPI_Group_rank ( MPI_Group group, int *rank );
```

- После завершения использования группа должна быть удалена

```
int MPI_Group_free ( MPI_Group *group );
```



Пример 1

```
int rank, i, size, size1, n;
MPI_Status status;
MPI_Group group, group1, group2;
int ranks[128], rank1, rank2, rank3;

MPI_Init(&argc, &argv);

MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_group(MPI_COMM_WORLD, &group);

size1 = size / 2;
for (i = 0; i < size1; i++) ranks[i] = i;

MPI_Group_incl(group, size1, ranks, &group1);
MPI_Group_excl(group, size1, ranks, &group2);
```



Пример 1

```
MPI_Group_rank(group1, &rank1);
MPI_Group_rank(group2, &rank2);

if (rank1 == MPI_UNDEFINED)
{
    if (!(size % 2) || rank != size - 1)
        MPI_Group_translate_ranks(group1, 1, &rank2, group,
&rank3);
    else
        rank3 = MPI_PROC_NULL;
}
else
    MPI_Group_translate_ranks(group2, 1, &rank1, group, &rank3);
```




Пример 1

```
MPI_Sendrecv(&rank, 1, MPI_INT, rank3, 1,  
             &n, 1, MPI_INT, rank3, 1,  
             MPI_COMM_WORLD, &status);
```

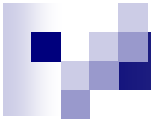
```
MPI_Group_free(&group);
```

```
MPI_Group_free(&group1);
```

```
MPI_Group_free(&group2);
```

```
cout << "process " << rank << ", n=" << n << endl;
```

```
MPI_Finalize();
```



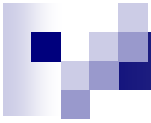
Коммуникаторы

- **Коммуникатор** – специально создаваемый служебный объект, объединяющий в своем составе группу процессов и ряд дополнительных параметров
- Коммуникаторы делятся на
 - **интракоммуникаторы** – используются для передачи данных внутри одной группы процессов
 - **интеркоммуникаторы** – предназначены для организации взаимодействия процессов, принадлежащих разным группам
- Коллективные операции выполняются только для процессов, принадлежащих одному интракоммуникатору



Интракоммуникаторы

- Все процессы программы объединены коммуникатором `MPI_COMM_WORLD`
- Для локального процесса создаётся коммуникатор `MPI_COMM_SELF`
- Константа `MPI_COMM_NULL` используется для обозначения ошибочного значения коммуникатора
- В процессе вычислений можно создавать и удалять коммуникаторы
- Один и тот же процесс может принадлежать разным коммуникаторам



Интракоммуникаторы

- Создание коммуникатора:

- дублирование уже существующего коммуникатора:

```
int MPI_Comm_dup ( MPI_Comm oldcom, MPI_comm *newcomm );
```

- создание нового коммуникатора из подмножества процессов существующего коммуникатора:

```
int MPI_comm_create ( MPI_Comm oldcom, MPI_Group group,  
MPI_Comm *newcomm );
```

- Операция создания коммуникаторов является коллективной и, тем самым, должна выполняться всеми процессами исходного коммуникатора

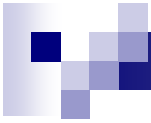


Пример 2

```
int rank, size, i, rbuf, ranks[128], new_rank;
MPI_Group group, new_group;
MPI_Comm new_comm;

MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_group(MPI_COMM_WORLD, &group);

for (i = 0; i < size / 2; i++) ranks[i] = i;
//создаём 2 группы процессов, каждый процесс входит только в 1
if (rank < size / 2)
    MPI_Group_incl(group, size / 2, ranks, &new_group);
else MPI_Group_excl(group, size / 2, ranks, &new_group);
```



Пример 2

//создаём коммунитор для каждой группы (всего будет создано два коммунитора для разных групп new_group)

```
MPI_Comm_create(MPI_COMM_WORLD, new_group, &new_comm);
```

//считаем сумму рангов для коммунитора, собираем сумму на всех процессах

```
MPI_Allreduce(&rank, &rbuf, 1, MPI_INT, MPI_SUM, new_comm);
```

//определяем ранг процесса в новой группе

```
MPI_Group_rank(new_group, &new_rank);
```

```
MPI_Comm_free(&new_comm); //удаляем коммуниторы
```

```
MPI_Group_free(&new_group); //удаляем связанные с ними группы
```

```
MPI_Finalize();
```



Одновременное создание нескольких коммунитаторов

```
int MPI_Comm_split ( MPI_Comm oldcomm, int split, int key,  
                    MPI_Comm *newcomm ),
```

где

- **oldcomm** – исходный коммунитатор,
- **split** – номер коммунитатора, которому должен принадлежать процесс,
- **key** – порядок ранга процесса в создаваемом коммунитаторе,
- **newcomm** – создаваемый коммунитатор



Пример 3

```
int rank, size, rank1;
MPI_Comm comm_revs;

MPI_Init(&argc, &argv);

MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

//создаём новый коммуникатор с обратным порядком процессов
MPI_Comm_split(MPI_COMM_WORLD, 1, size - rank, &comm_revs);

MPI_Comm_rank(comm_revs, &rank1); //ранг в новом коммуникаторе

cout << "rank = " << rank << " rank1 = " << rank1 << endl;

MPI_Comm_free(&comm_revs); //уничтожаем коммуникатор
MPI_Finalize();
```




Пример 4 (модификация примера 2)

```
int rank, size;
int new_rank, rbuf;
MPI_Comm new_comm;
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
int div = rank / (size / 2);
//создаём новые коммуникаторы как в примере 2
MPI_Comm_split(MPI_COMM_WORLD, div, rank, &new_comm);
MPI_Comm_rank(new_comm, &new_rank); //ранг в новом коммуникаторе
MPI_Allreduce(&rank, &rbuf, 1, MPI_INT, MPI_SUM, new_comm); //считаем
сумму рангов для коммуникатора, собираем сумму на всех процессах
MPI_Comm_free(&new_comm); //уничтожаем коммуникатор
cout << "rank = " << rank << " newrank = " << new_rank << " rbuf = "
<< rbuf << endl;
MPI_Finalize();
```



Пример 5

//Подготавливаем данные и проверяем, что число процессов – полный квадрат

```
int div = rank / p, mod = rank % p; //декартовы координаты процесса
```

```
//создаём новые коммуниторы для строк и столбцов
```

```
MPI_Comm_split(MPI_COMM_WORLD, div, mod, &row_comm);
```

```
MPI_Comm_split(MPI_COMM_WORLD, mod, div, &col_comm);
```

```
MPI_Comm_rank(row_comm, &row_rank); //ранг в строке
```

```
MPI_Comm_rank(col_comm, &col_rank); //ранг в столбце
```

```
//считаем сумму рангов в строках и столбцах
```

```
MPI_Allreduce(&rank, &row_sum, 1, MPI_INT, MPI_SUM, row_comm);
```

```
MPI_Allreduce(&rank, &col_sum, 1, MPI_INT, MPI_SUM, col_comm);
```

```
MPI_Comm_free(&row_comm); //уничтожаем коммунитор
```

```
MPI_Comm_free(&col_comm); //уничтожаем коммунитор
```

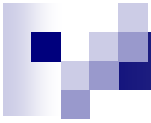
```
//завершаем выполнение программы
```



Интракоммуникаторы

- Вызов функции `MPI_Comm_split` должен быть выполнен в каждом процессе коммуникатора `oldcomm`
- Процессы разделяются на непересекающиеся группы с одинаковыми значениями параметра `split`. На основе сформированных групп создается набор коммуникаторов. Порядок нумерации процессов соответствует порядку значений параметров `key` (процесс с большим значением параметра `key` будет иметь больший ранг)
- После завершения использования коммуникатор должен быть удален:

```
int MPI_Comm_free ( MPI_Comm *comm );
```



Виртуальные топологии

- Под **топологией** вычислительной системы понимают структуру узлов сети и линий связи между этими узла. Топология может быть представлена в виде графа, в котором вершины есть процессоры (процессы) системы, а дуги соответствуют имеющимся линиям (каналам) связи.
- Парные операции передачи данных могут быть выполнены между любыми процессами коммутатора, в коллективной операции принимают участие все процессы коммутатора. Следовательно, логическая топология линий связи между процессами в параллельной программе имеет структуру полного графа.
- Возможно организовать логическое представление любой необходимой виртуальной топологии. Для этого достаточно сформировать тот или иной механизм дополнительной адресации процессов



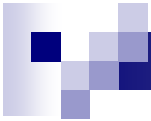
Декартовы топологии (решетки)

- Множество процессов представляется в виде прямоугольной решетки, а для указания процессов используется декартова система координат
- Для создания декартовой топологии предназначена функция:

```
int MPI_Cart_create( MPI_Comm oldcomm, int ndims, int *dims,  
                    int *periods, int reorder, MPI_Comm *cartcomm);
```

где:

- **oldcomm** - исходный коммуникатор,
- **ndims** - размерность декартовой решетки,
- **dims** - массив длины **ndims**, задает количество процессов в каждом измерении решетки,
- **periods** - массив длины **ndims**, определяет, является ли решетка периодической вдоль каждого измерения,
- **reorder** - параметр допустимости изменения нумерации процессов,
- **cartcomm** – создаваемый коммуникатор с декартовой топологией процессов.



Декартовы топологии (решетки)

- Для определения декартовых координат процесса по его рангу можно воспользоваться функцией:

```
int MPI_Card_coords( MPI_Comm comm, int rank,  
                    int ndims, int *coords ),
```

где:

- ☐ **comm** – коммуникатор с топологией решетки,
 - ☐ **rank** – ранг процесса, для которого определяются декартовы координаты,
 - ☐ **ndims** – размерность решетки,
 - ☐ **coords** – возвращаемые функцией декартовы координаты процесса.
-
- Обратное действие – определение ранга процесса по его декартовым координатам – обеспечивается при помощи функции:

```
int MPI_Cart_rank( MPI_Comm comm, int *coords, int *rank ),
```



Декартовы топологии (решетки)

- Процедура разбиения решетки на подрешетки меньшей размерности обеспечивается при помощи функции:

```
int MPI_Cart_sub( MPI_Comm comm, int *subdims,  
                  MPI_Comm *newcomm ),
```

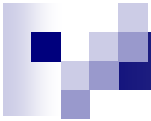
где:

- ☐ **comm** - исходный коммуникатор с топологией решетки,
 - ☐ **subdims** – массив для указания, какие измерения должны остаться в создаваемой подрешетке,
 - ☐ **newcomm** - создаваемый коммуникатор с подрешеткой.
-
- В ходе своего выполнения функция **MPI_Cart_sub** определяет коммуникаторы для каждого сочетания координат фиксированных измерений исходной решетки



Пример 6

```
int div = rank / p, mod = rank % p;
dims[0] = dims[1] = p;
//Создаём двумерную решётку-тор размера p x p
MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 0, &cart_comm);
MPI_Cart_coords(cart_comm, rank, 2, coords);
//создаём новые коммуникаторы для строк и столбцов
subdims[0] = 1, subdims[1] = 0;
MPI_Cart_sub(cart_comm, subdims, &row_comm);
subdims[0] = 0, subdims[1] = 1;
MPI_Cart_sub(cart_comm, subdims, &col_comm);
MPI_Comm_rank(row_comm, &row_rank); //ранг в строке
MPI_Comm_rank(col_comm, &col_rank); //ранг в столбце
//считаем сумму рангов в строках и столбцах
MPI_Allreduce(&rank, &row_sum, 1, MPI_INT, MPI_SUM, row_comm);
MPI_Allreduce(&rank, &col_sum, 1, MPI_INT, MPI_SUM, col_comm);
MPI_Comm_free(&row_comm); //уничтожаем коммуникатор
MPI_Comm_free(&col_comm); //уничтожаем коммуникатор
```

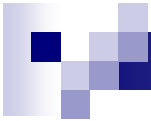
Декартовы топологии (решетки)

- Функция `MPI_Cart_shift` обеспечивает получение рангов процессов, с которыми текущий процесс (вызвавший функцию) должен выполнить обмен данными:

```
int MPI_Cart_shift( MPI_Comm comm, int dir, int disp,  
                   int *source, int *dst),
```

где:

- **comm** – коммуникатор с топологией решетки,
- **dir** – номер измерения, по которому выполняется сдвиг,
- **disp** – величина сдвига (<0 – сдвиг к началу измерения),
- **source** – ранг процесса, от которого должны быть получены данные,
- **dst** – ранг процесса которому должны быть отправлены данные



Топология графа

- Создание коммуникатора с топологией типа граф:

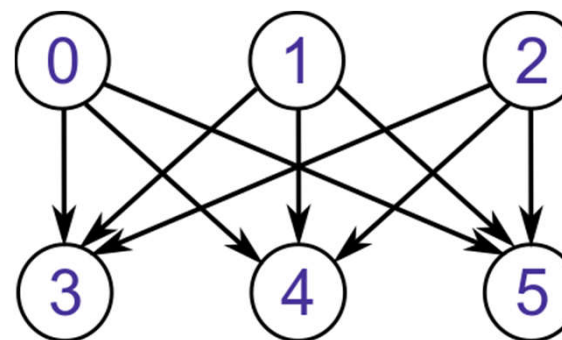
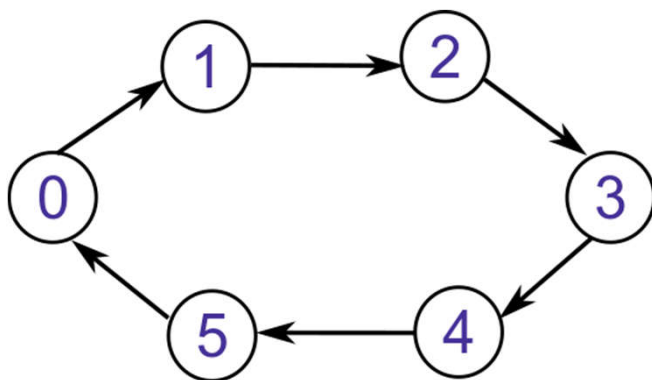
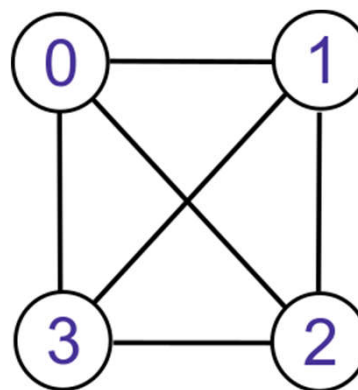
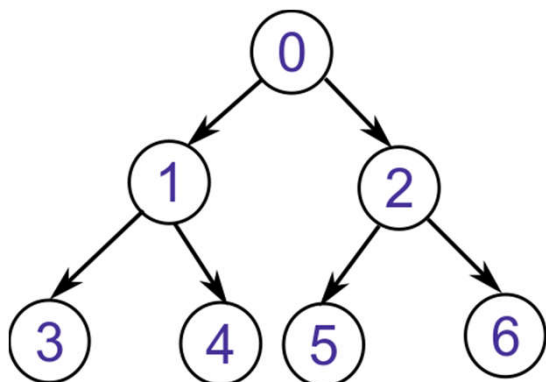
```
int MPI_Graph_create( MPI_Comm oldcomm, int nnodes,  
                     int *index, int *edges,  
                     int reorder, MPI_Comm *graphcomm ),
```

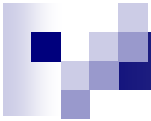
где:

- ☐ **oldcomm** – исходный коммуникатор,
 - ☐ **nnodes** – количество вершин графа,
 - ☐ **index** – количество исходящих дуг для каждой вершины,
 - ☐ **edges** – последовательный список дуг графа,
 - ☐ **reorder** – параметр допустимости изменения нумерации процессов,
 - ☐ **cartcomm** – создаваемый коммуникатор с топологией типа граф.
- Операция создания топологии является коллективной и, тем самым, должна выполняться всеми процессами исходного коммуникатора.

Задачи

- Составить массивы степеней и рёбер для следующих графов





Топология графа

- Количество соседних процессов, в которых от проверяемого процесса есть выходящие дуги, может быть получено при помощи функции:

```
int MPI_Graph_neighbors_count( MPI_Comm comm, int rank,  
                               int *nneighbors );
```

где

- **nneighbors** – количество выходящих дуг из вершины ранг **rank**

- Получение рангов соседних вершин обеспечивается функцией:

```
int MPI_Graph_neighbors( MPI_Comm comm, int rank,  
                         int neighbors,  
                         int *mneighbors );
```

где

- **neighbors** – количество дуг, выходящих из вершины ранга **rank**
- **mneighbors** – массив размера **neighbors**



Пример 7

```
int rank, rank1, i, size, num;
MPI_Status status;
MPI_Comm comm_graph;
vector<int> index, edges, neighbors;
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
index.resize(size);
edges.resize(size);
for (i = 0; i < size; i++) index[i] = size + i - 1;
for (i = 0; i < size - 1; i++) {
    edges[i] = i + 1;
    edges[size + i - 1] = 0;
}

MPI_Graph_create(MPI_COMM_WORLD, size, index.data(), edges.data(), 1,
&comm_graph);
```



Пример 7

```
//Количество соседей процесса
MPI_Graph_neighbors_count(comm_graph, rank, &num);
//выделяем память под массив рангов соседей и получаем ранги соседей
neighbors.resize(num);
MPI_Graph_neighbors(comm_graph, rank, num, neighbors.data());

for (i = 0; i < num; i++) {
    MPI_Sendrecv(&rank, 1, MPI_INT, neighbors[i], 1, &rank1, 1,
MPI_INT, neighbors[i], 1, comm_graph, &status);
    cout << "process " << rank << " communicate with process " <<
rank1 << endl;
}

MPI_Finalize();
```